

Raspberry Pi Zero 2 W 引き継ぎ資料（16期作成）

1. 環境構築

1.1 環境構築の準備

ハードウェア

- Raspberry Pi Zero 2 W
- microSDカード（16GB以上あれば良い）
- 電源（5V / 2.5A）（PCからのUSB給電は電力不足で突然落ちる原因になるためNG。必ずコンセントやモバイルバッテリーから給電する）
- Micro USBケーブル（電源に接続する）
- PC（microSDカードのスロットがない場合は、USBの**SDカードリーダー**などが必要）

[補足1] ピンから直接の電源供給はリスクが大きいので、テスト環境ではなるべくMicro USB ケーブルから給電する

[補足2] モニターやMini HDMI ケーブルは、最初のWi-Fi接続に失敗したとき原因がわからないというデメリットはあるものの、SDカードを焼き直せばいいので実質的に不要

ソフトウェア

- Raspberry Pi Imager
- Visual Studio Code（VS Code）

1.2 Raspberry Pi Zero 2 W のセットアップ

1.2.1 OS イメージの準備

1. PCにmicroSDカードを挿入し、認識させておく

[補足] 過去にラズパイ等で使用したSDカードをWindows PCに挿すと、容量が極端に少なく表示されたり、「フォーマットする必要があります」という警告が出たりするが、これはWindowsがLinuxのデータ形式を読み込めないだけの正常な挙動。警告は無視し、そのままRaspberry Pi Imagerを開いて正しい容量が認識されていれば問題ない。

2. Raspberry Pi Imager をインストールする（<https://www.raspberrypi.com/software/>）。セットアップでは特に変更は不要
3. 起動後、
 - **Device** → *Raspberry Pi Zero 2 W*
 - **OS** → *Raspberry Pi OS(other)*から*Raspberry Pi OS Lite (64-bit)*（GUIは不要）
 - **Storage** → MicroSDカード
4. 以下を設定・有効化する

- **Hostname**（ネットワーク上におけるラズパイの名前）：短くて分かりやすい、すべて英小文字の名前
- **Localisation**（国・キーボード設定）：首都を **Tokyo (Japan)**、タイムゾーンを **Asia/Tokyo**、キーボードを **jp** にする
- **User**（アカウント）：ログインに使う。ユーザー名はすべて英小文字、パスワードは任意
- **Wi-Fi**：Raspberry Pi Zero 2 W は 5GHz帯のWi-Fiには非対応のため、必ず 2.4GHz帯のSSIDを設定する
- **Remote access**（遠隔操作）：SSHを有効化し、**パスワード認証を使う** を選択
- **Raspberry Pi Connect**：オフのままでよい

[重要] Hostname、ユーザー名、パスワードは後で使うのでメモに残しておく

5. Writing（書き込み）を実行

1.2.2 初回起動と接続

1. 書き込みが終わった microSD をラズパイに挿入する
2. Micro USBケーブルをラズパイの **PWR IN のポート** に挿して電源投入
3. ラズパイの緑色のLEDが点滅し始めるので、そのまま1〜2分ほど待機する

[補足] 初回起動時は、OSの初期設定やWi-Fiへの接続処理が行われるため少し時間がかかる。LEDの点滅が落ち着くまで待つこと。

4. PC側でターミナル（WindowsではコマンドプロンプトやPowerShell、Macの場合はターミナル）を開き、次のコマンドで接続

```
ssh ユーザー名@ホスト名.local
```

（例：ユーザー名が **pi**、Hostnameが **cansat** の場合は **ssh pi@cansat.local**）

[補足] 初回接続時のみ **Are you sure you want to continue connecting (yes/no/[fingerprint])?** と聞かれるので、**yes** と入力

5. パスワードを求められるので、Imager で設定したものを使用

[補足] パスワード入力時は画面上に文字が一切表示されない。入力されていないように見えるが、内部ではちゃんと打ち込まれているので、気にせずパスワードを打ち込んでEnterを押す。

6. 緑色の文字で **ユーザー名@ホスト名:~ \$** のような文字が出れば、ラズパイへのログイン成功

1.2.3 接続できない場合は以下を確認

1.2.3.1 ホスト名が見つからない・タイムアウトになる場合

WindowsやAndroid端末では、mDNS（.local）によるホスト名解決が安定的にサポートされておらず、ホスト名接続は一般に不安定。

1. 同一ネットワークにいるか確認

[補足] PCが有線LAN接続であっても、ラズパイと同じWi-Fiルーターに繋がっていれば問題ない。

2. IPアドレスで直接接続する

`.local` で繋がらない場合は、ルーターの管理画面やスマホのネットワークスキャンアプリ等でラズパイの IPアドレス を確認して、ホスト名のところを IP に置き換えて接続する

```
ssh ユーザー名@IPアドレス
```

1.2.3.2 Permission denied (publickey,password). と弾かれる場合

ラズパイ側のSSH設定でパスワード認証が無効になっている可能性があるため、以下の手順で設定を修正する。

1. まず、以下のコマンドでパスワード認証を強制的に指定して接続する

```
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no ユーザー名@ラズパイのIPアドレス（またはホスト名.local）
```

2. ログインできたら、nanoエディタでラズパイ上のSSH設定ファイルを開く

```
sudo nano /etc/ssh/sshd_config
```

3. ファイル内から `PasswordAuthentication` の行を探し、パスワード認証を有効（yes）にする

- `PasswordAuthentication no` になっていたら `yes` に変更する
- `#PasswordAuthentication yes` のように `#` でコメントアウトされている場合は、先頭の `#` を削除する

[補足] nanoエディタの操作方法は付録を参照（準備中...）。

4. 設定を変更してエディタを閉じたら、SSH サーバーを再起動して設定を反映させる

```
sudo systemctl restart ssh
```

1.2.4 初期アップデート

初回起動時は、OSとパッケージを最新の状態にしておく。

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

[補足] `sudo reboot` を実行するとラズパイが再起動するため、PC側のSSH接続は強制的に切断される。

1.3 VS Code によるSSH接続したラズパイのターミナル操作

今後の作業を効率化するため、VS Codeから直接ラズパイを操作・編集できるように設定する。以後、ターミナル操作はここで行うことができる。

1.3.1 VS Code 拡張機能のインストール

PCで VS Code を開き、拡張機能「Remote - SSH」をインストールする

1.3.2 ラズパイへの接続

1. VS Code 左下の「><」アイコンをクリック
2. 画面上部に出るメニューから `Remote-SSH: Connect to Host...` を選択
3. `+ Add New SSH Host...`（新規 SSH ホストを追加する...）を選択し、sshコマンドを実行

```
ssh ユーザー名@ホスト名.local（あるいはIPアドレス）
```

4. 「更新する SSH 構成ファイル」を聞かれたら、一番上の標準パス（Windowsなら `C:\Users\ユーザー名\.ssh\config`）を選択
5. 右下に「Host added!」（ホストが追加されました!）と通知が出たら `Connect` をクリック
6. Platform（ターゲットのOS）を聞かれたら `Linux` を選び、パスワードを求められたら入力

[補足] 初回接続時は、ラズパイ側に通信用プログラム「VS Code Server」が自動インストールされるため、1〜2分ほど時間がかかる。

7. 接続が完了すると、VS Code 左下のステータスバーが `>< SSH: ホスト名.local` 表示になる

1.3.3 フォルダの展開

1. 左側のエクスプローラーアイコンから `Open Folder`（フォルダを開く）をクリック
2. ラズパイのホームディレクトリ（`/home/ユーザー名` など初めから入力されているパス）をそのまま選択してOK
3. 再度パスワードを聞かれるので入力すると、ラズパイ内のファイルが一覧表示され、PCと同じ感覚でファイルの編集・保存ができるようになる。

1.3.4 ターミナルの起動

新しいターミナル (**Ctrl + Shift + @** や、上部メニュー「ターミナル」から) を開くと、画面下部にラズパイのターミナル (~\$) が表示される。以降のコマンド操作はすべてここで行う。

[補足] ターミナルは Pi のユーザ権限で動いているため、システムの設定変更など (root権限が必要な操作) を行う場合は、これまで通りコマンドの先頭に **sudo** をつける。

1.3.5 トラブルシューティング

Raspberry Pi Zero 2 W はメモリが非常に少ないため、初回接続時の「VS Code Server」のダウンロードおよびインストール処理において、フリーズやタイムアウトを起こすことが頻繁にある。その場合は、以下の2つの対策を実施してから再度接続を試みる。

対策1：VS Codeのタイムアウト時間を延ばす（PC側の設定）

標準設定では、ラズパイからの応答が15秒ないとVS Codeが勝手に接続を諦めてしまう。Zero 2 Wの処理を待てるように設定を変更する。

1. **Ctrl + ,** や左下の歯車アイコンから「設定 (Settings)」を開く
2. 検索バーに **Remote.SSH: Connect Timeout** と入力する
3. 値を **15** から **60** に変更する (入力すると自動で保存される)

対策2：仮想メモリ (Swap) を一時的に増設する（ラズパイ側の設定）

メモリ不足によるOSのフリーズを防ぐため、Linuxの標準機能を使ってSDカードの容量を一時的に1GBの仮想メモリとして借用する。

1. PC標準のターミナル (PowerShell等) から **ssh** コマンドでラズパイにログインする
2. 過去の接続失敗で壊れたVS Codeの残骸データが残っている場合があるため、以下のコマンドで掃除する

```
rm -rf ~/.vscode-server
```

3. 以下のコマンドを順番に実行し、1GBの仮想メモリを作成・有効化する

```
sudo fallocate -l 1G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
```

4. エラーが出ずに完了したら、**exit** でターミナルを閉じ、再度 VS Code から接続を試みる

[補足] この仮想メモリは、ラズパイを再起動 (**sudo reboot** やシャットダウン) すると自動的にリセットされ、元の状態 (512MB) に戻る。

1.4 ラズパイの基本的な運用

1.4.1 2回目以降の起動と接続

1. ラズパイにMicro USBケーブル（電源）を挿し、給電する
2. 緑色のLEDが激しく点滅し始めるので、Wi-Fiに自動接続されて点滅が落ち着くまで1〜2分待機する
3. PCでVS Codeを開き、左下の「><」アイコンをクリック
4. `Connect to Host...` を選択し、登録されているホスト名（またはIPアドレス）をクリック
5. パスワードを入力

1.4.2 作業の終了とシャットダウン

ラズパイは内部でOSが動いているため、いきなり電源ケーブルを抜くのは厳禁。必ず以下の手順で安全に電源を落とす。

1. VS Codeのターミナルで以下のコマンドを実行

```
sudo poweroff
```

[補足] `sudo shutdown -h now` でも可。

2. 接続が自動的に切断される

- VS Code上に `Cannot reconnect.`（接続が切断されました）というメッセージが出る
- ラズパイ本体の緑色のLEDが何度か点滅したあと、完全に消灯する

[補足] PCのターミナルで行った場合は `Connection closed by ...` のようなメッセージが表示される

3. LEDの反応が完全になくなったことを確認してから、Micro USBケーブルを抜く

[補足1] ラズパイの電源を切らずに、PC側だけ接続を解除したい場合は、`exit` コマンドを実行すればよい。

[補足2] microSDカードはラズパイに挿しっぱなしでよい。

2. ハードウェア制御の準備

2.1 シリアル通信 / GPIO / I2C の有効化

`raspi-config` で設定を行う。

1. ターミナルで以下のコマンドを実行

```
sudo raspi-config
```

2. 以下を有効化

- **I2C**: `Interface Options` → `I2C` → `<Yes>`
- **Serial**: `Interface Options` → `Serial Port` を選択すると2つ質問されるので、以下のように答える
 1. `Would you like a login shell to be accessible over serial?` (シリアル通信経由でのログインを許可するか?) → `<No>`
[重要] ここをYesにするとOSがポートを占領してしまい、GPS等のモジュールと干渉するため**必ずNoにする**。
 2. `Would you like the serial port hardware to be enabled?` (ハードウェアのシリアルポートを有効にするか?) → `<Yes>`
- 3. 完了後、`<Finish>` を選ぶと「Would you like to reboot now? (今すぐ再起動しますか?)」と聞かれるので、`<Yes>` を選んで自動再起動させる。

[補足1] 再起動が始まるとラズパイとの通信が切れる。VS Code を一度閉じ、ラズパイ本体の緑LEDの点滅が落ち着いてから再度開き直す。

[補足2] もし自動再起動の画面が出なかった場合のみ、手動で `sudo reboot` を実行する。

2.2 Python 実行環境の構築

2.2.1 依存パッケージのインストール

使用するセンサーやモジュールに合わせて、必要なコマンドを選択して実行する。

```
# 1. 【必須】 ソースコード管理 (git) や、SSH切断時のプログラム継続実行 (tmux) 、Pythonパッケージ管理 (pip) に必須のため必ず実行する。
```

```
sudo apt install -y git tmux python3-pip python3-setuptools
```

```
# 2. 【GPIO】 LEDやモーター、各種スイッチなどを制御する場合
```

```
sudo apt install -y python3-gpiozero python3-rpi-lgpio liblgpio1
```

```
# 3. 【I2C通信】 気圧センサーや9軸センサーなどを接続する場合
```

```
sudo apt install -y i2c-tools python3-smbus
```

```
# 4. 【シリアル通信】 GPSモジュールや無線機を接続する場合
```

```
sudo apt install -y python3-serial
```

[重要：GPIO] 過去に主流だった **RPi.GPIO** は、最新OS (Bookworm以降) では **非推奨** である。本プロジェクトでは、公式推奨であり安全に動作する **gpiozero** (およびバックエンドとしての `rpi-lgpio`) を使用する。

[補足] 参考のために正常にインストールできているか確かめられるコマンドを以下に記しておく。

```
# git / tmux / pip / i2c-tools(i2cdetect) → バージョン情報が出ればOK
git --version
```

```
tmux -V
python3 -m pip --version
i2cdetect -V

# setuptools / gpiozero / rpi-lgpio / liblgpio1 / smbus / serial → print内のメッセ
ー지가出ればOK
# もしどれか一つでもインストールに失敗していれば、ModuleNotFoundError という赤いエラーが
出る
python3 -c "import setuptools; import gpiozero; import lgpio; import RPi.GPIO;
import smbus; import serial; print('All modules OK')"
```

2.2.2 Python 仮想環境の作成

ラズパイOS（Bookworm以降）ではシステム全体への `pip install` が禁止されているため、必ず仮想環境を作成してその中で作業する。

```
# 1. 仮想環境 "venv" の作成（システムパッケージを引き継ぐ --system-site-packages が重
要）
# これにより、aptで入れた gpiozero や smbus 等のハードウェア制御ツールが仮想環境内でもそ
のまま使用可能になる
python3 -m venv --system-site-packages venv

# 2. 仮想環境の有効化（ターミナルの左端に (venv) と表示されれば成功）
source venv/bin/activate

# 3. pip自体の更新
pip install --upgrade pip

# その他、センサー類で必要なライブラリがあればここで追加
# pip install "---"
```

[重要] ラズパイを再起動したりSSHをつなぎ直したりした後は仮想環境が外れているため、Pythonのプログラムを実行する前には必ず `source venv/bin/activate` を実行して **(venv)** の状態にする。

[補足1] 16期では、GPS(micropyGPS)や9軸(BNO055)、気圧(BMP180)などの制御スクリプトは、pipでの個別インストールではなく `sensor/` フォルダ内に配置したスクリプトを直接 `import` して使用している。詳細は2.3節に記載。

[補足2] 仮想環境から抜きたい場合は `deactivate` を実行すればよい。

2.3 Gitリポジトリの運用

これは16期のNSE2026における運用例（チーム八木山）であるので、フォルダ名や構成に合わせて適宜コマンドなど変更すること。Git及びGitHubについては別資料を参照のこと（準備中...）。

2.3.1 リポジトリの作成と構成

新規作成しない場合であれば不要。

1. GitHubにおいてリポジトリを作成する
2. PC上のローカル環境にリポジトリをクローンする
3. 好きにリポジトリの構成を行う。16期の場合、

2.3.2 リポジトリのクローン

ラズパイで以下のコマンドを実行し、リポジトリをクローンする

```
# git clone https://github.com/(GitHubのユーザー名)/(リポジトリ名)
git clone https://github.com/jumonjiakimasap2-cell/NSE2026
```

2.3.3 リポジトリの運用

10. 付録

10.1 nanoエディタの操作方法

10.2

以下未編集

2. ハードウェア制御の準備

2.2 Python 実行環境の構築

2.2.1 依存パッケージのインストール

```
# 4. GPS解析用ライブラリ
pip install pynmea2

# カメラ / 画像処理 (Picamera2 + OpenCV + NumPy)
sudo apt install -y python3-picamera2 python3-libcamera libcamera-apps python3-opencv python3-numpy

# OpenCVでGUI表示 (imshow等) する場合に必要。ヘッドレス運用なら不要なことが多い
sudo apt install -y libgl1

# ===== ここから下は自前ビルドをするなら必要=====

# C拡張やライブラリをソースからビルドする場合に必要
# sudo apt install -y build-essential python3-dev swig

# lgpio を C で開発/コンパイルする場合のヘッダ (Pythonで動かすだけなら通常不要)
# sudo apt install -y liblgpio-dev
```

6. リポジトリのクローン

```
git clone https://github.com/YutakaOkutani/TRC2026
cd TRC2026
```

6.1 ドキュメントの役割分担（重複を避けるため）

- `README.md`
 - セットアップ手順、実行コマンド、運用手順。
- `csmn/arch_summary.md`
 - `csmn/` の設計内容と保守ルールのみ。
- `runs/cam/cam_relay_readme.md`
 - カメラ中継テスト（SBC↔PC）の手順のみ。

6.2 実行コマンド早見表

前提:

- 作業ディレクトリは `~/TRC2026`
- 必要なら先に仮想環境を有効化: `source venv/bin/activate`

```
# 本番実行
python3 main.py

# フェーズ限定オーケストレーション
python3 runs/orch/orch_p1_p3.py
python3 runs/orch/orch_p2_p3.py
python3 runs/orch/orch_p3_p4.py
python3 runs/orch/orch_p4_p7.py
python3 runs/orch/orch_p1_p7.py
python3 runs/orch/orch_p2_p7.py

# 各種テストコード
python3 runs/diag/sensor_diag.py
python3 runs/diag/gps_diag.py
python3 runs/diag/motor_diag.py
python3 runs/diag/led_diag.py

# 審査書試験系（フェーズ0試験）
python3 runs/evt/open_parachute.py
python3 runs/evt/landing_impact.py

# 画像・カメラ系
python3 runs/cam/cam_capture_data.py --count 10 --interval 0.5 --prefix sample
python3 runs/cam/cam_detector_dbg.py --phase 4
```

```
# ログ解析（PC上で実行）  
python3 anlz/log_anlz.py
```

7. カメラの設定

カメラの初期設定

```
# 設定ファイルを編集（OV5647を明示する場合）  
sudo nano /boot/firmware/config.txt  
  
# 以下を追加または修正（行がある場合は値を合わせる）  
camera_auto_detect=0  
dtoverlay=ov5647
```

```
# 編集後、再起動  
sudo reboot
```

カメラ認識の確認

```
# 認識デバイスの一覧  
rpicas-hello --list-cameras  
  
# dmesg による初期化ログ確認  
sudo dmesg | grep -Ei "ov5647|camera|unicam" | tail -n 30  
  
# 初期化時間の確認  
time rpicas-hello -t 1
```

カメラコマンドの確認

```
rpicas-hello --help  
rpicas-still --help  
rpicas-vid --help
```

カメラ映像のテスト

```
# ライブプレビュー  
rpicas-hello -t 0  
  
# 静止画撮影
```

```
rpicam-still -o test.jpg

# 動画撮影
rpicam-vid -t 10000 -o test.h264

# 高解像度での静止画撮影テスト
rpicam-still --width 2592 --height 1944 -o maxres.jpg
```

8. 実行（テスト時）

本番用コード

```
# ===== 推奨：最初からデタッチ状態で起動する方法 =====
# このコマンドで tmux セッション内に main.py を起動する。SSH が切れても tmux セッション
# が残るので実行継続できる
# ※ 重要: `python3 main.py` を tmux の外で起動すると、SSH切断時に一緒に止まる可能性が
# ある
# ※ 重要: これは「SSH切断対策」。ラズパイの再起動/電源断まで含めて継続したい場合は、下の
# systemd 設定を使う
cd ~/TRC2026
tmux new-session -d -s cansat 'bash -lc "source venv/bin/activate && exec python3
main.py"'

# ログ/画面を確認したいときに接続（アタッチ）
tmux attach -t cansat

# 画面を閉じずに離脱（デタッチ）する操作
# キー操作: Ctrl+b を押してから d
# ※ `exit` / Ctrl+C は tmux セッション内のシェル/プログラムを終了させるので注意

# セッション一覧を確認（起動確認に使える）
tmux ls

# すでに cansat セッションが存在する場合は、新規作成せず接続して再利用
# tmux attach -t cansat
```

```
# ===== 対話的に起動する方法（手動操作したい場合） =====
cd ~/TRC2026
tmux new -s cansat
source venv/bin/activate
python3 main.py
# 離脱時は Ctrl+b → d
# 再接続後は `tmux attach -t cansat`
```

9. トラブルシューティング

センサが認識されない

- `sudo i2cdetect -y 1` で確認
- 配線の導通を確認

10. 本番向けの設定

本番運用 (systemd) : 電源投入で自動起動し、SSH切断後も継続実行する手順

`tmux` は「手動起動して画面を見ながらデバッグ/運用する」ために便利。

本番運用（電源投入で自動起動・SSH切断の影響を受けない・異常終了時に自動復帰）には `systemd` を使う。

1. 前提確認（パスを固定する）

`systemd` は相対パスに弱いので、先に **実際の絶対パス** を確認する。

```
cd ~/TRC2026
pwd
which python3
ls venv/bin/python
```

想定例（環境に合わせて読み替える）

- リポジトリ: `/home/pi/TRC2026`
- venv の Python: `/home/pi/venv/bin/python`

2. サービスファイルの作成

```
sudo nano /etc/systemd/system/cansat.service
```

3. 設定内容の書き込み（推奨例）

```
[Unit]
Description=CanSat Main Mission Script
# ネットワークを使う処理（通知・通信など）がある場合に備えて、ネットワーク起動後に開始する
# Wants=network-online.target
# After=network-online.target

[Service]
Type=simple
User=pi
Group=pi

# ここは実際のクローン先に合わせて必ず書き換える
WorkingDirectory=/home/pi/TRC2026
```

```
# ログを journald に即時反映しやすくする (print が遅延しにくい)
Environment=PYTHONUNBUFFERED=1

# venv を使う場合は venv の python を使う (activate は不要)
# ※ パスは必ず実環境に合わせる
ExecStart=/home/pi/TRC2026/venv/bin/python /home/pi/TRC2026/main.py

# 異常終了時に自動再起動 (ミッション継続のため重要)
Restart=on-failure
RestartSec=5

# 手動停止時の扱いを安定させるための猶予
TimeoutStopSec=15

# 標準出力/標準エラーは journalctl で確認する
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

補足 (重要)

- `WorkingDirectory` と `ExecStart` のパスがズレると起動失敗するので、必ず実際の環境に合わせて書き換えること
- `venv` を使う場合、`source venv/bin/activate` は不要。`ExecStart` に `venv` の Python を直接書くのが `systemd` の定石
- `Restart=on-failure` はクラッシュ時に再起動し、`sudo systemctl stop cansat.service` のような手動停止時は再起動しないので運用しやすい
- 通信を使わない構成なら `network-online.target` は必須ではないが、将来の通知機能などを考えると入れておく方が無難

4. `cansat.timer` の作成 (起動から5分後に開始する) (任意)

電源投入直後ではなく、**起動から5分後**に `cansat.service` を開始したい場合は、`timer` を使う。

```
sudo nano /etc/systemd/system/cansat.timer
```

```
[Unit]
Description=Start CanSat mission 5 minutes after boot

[Timer]
# systemd 起動 (=OS起動) から5分後に cansat.service を実行
OnBootSec=5min

# この timer が起動する対象ユニット
Unit=cansat.service
```

```
# タイミングの揺れを小さくしたい場合（任意）
AccuracySec=1s

[Install]
WantedBy=timers.target
```

補足

- `cansat.timer` は「いつ起動するか」を担当し、実際の処理本体は `cansat.service` が担当する
- 自動起動の有効化は `cansat.service` ではなく `cansat.timer` に対して行う（`service` は `timer` から呼ばれる）

5. サービス / タイマーの有効化

```
# 設定の反映
sudo systemctl daemon-reload

# 起動時に自動起動 + 今すぐタイマーを開始（5分カウント開始）
sudo systemctl enable --now cansat.timer
```

初回の動作確認で「5分待たずにすぐ実行したい」場合だけ、手動でサービスを起動してよい。

```
sudo systemctl start cansat.service
```

6. 状態・ログの確認（必須）

```
# タイマーが有効か確認（NEXT に次回実行予定が出る）
sudo systemctl list-timers cansat.timer
sudo systemctl status cansat.timer
```

```
# 稼働状態の確認（active (running) になっているか）
sudo systemctl status cansat.service
```

```
# 直近ログを表示
sudo journalctl -u cansat.service -e
```

```
# リアルタイムでログを追う（デバッグ時に便利）
sudo journalctl -u cansat.service -f
```

7. 運用で使う基本コマンド

```
# 再起動（コード更新後など）
sudo systemctl restart cansat.service

# タイマーの5分カウントを今この瞬間からやり直したい場合
sudo systemctl restart cansat.timer

# 停止
sudo systemctl stop cansat.service

# 自動起動（5分遅延起動）の停止
sudo systemctl stop cansat.timer

# 自動起動の無効化（必要時のみ）
sudo systemctl disable cansat.timer

# 自動起動設定の確認
automatically_enabled=$(sudo systemctl is-enabled cansat.timer); echo
$automatically_enabled
```

8. 本当に「電源投入から5分後に自動起動」するか確認（重要）

```
sudo reboot
```

再起動後に SSH 接続して、以下を確認する。

```
sudo systemctl status cansat.timer
sudo systemctl list-timers cansat.timer
sudo systemctl status cansat.service
sudo journalctl -u cansat.service -b
```

`-b` は「今回の起動（boot）分のログだけ」を見るためのオプション。

9. よくある起動失敗ポイント（先に潰す）

- パス違い: `WorkingDirectory` / `ExecStart` が実際の配置と違う
- venv 未作成: `/home/pi/TRC2026/venv/bin/python` が存在しない
- 権限問題: `User=pi` でアクセスできないファイル/ディレクトリがある
- ライブラリ不足: 手動実行では動いたが、`venv` 側に必要ライブラリが入っていない
- 例外で即終了: `sudo journalctl -u cansat.service -e` で Python の traceback を確認
- timer の有効化漏れ: `cansat.service` ではなく `cansat.timer` を `enable` しているか確認

10. tmux との使い分け（整理）

- **tmux**: 手動起動・画面確認・その場のデバッグ向け
- **systemd**: 本番常駐・自動起動・異常終了時の自動復帰向け

本番中に一時的に手動デバッグしたい場合は、先に **sudo systemctl stop cansat.service** してから **tmux** で起動する（同時起動を避ける）。

11. 便利なコマンドや設定

基本的なgit操作コマンド

```
# ファイルをステージングに追加
git add .
# コミットを作成
git commit -m "Initial commit"
# GitHub へ初回プッシュ
git push -u origin main
# 2回目以降
git push
```

```
# ローカルを GitHub の最新版で完全に上書きするコマンド
git fetch origin
git reset --hard origin/main
# 一行で実行するコマンド
git fetch origin && git reset --hard origin/main
```

```
# ローカルの変更を残しつつ、GitHub の更新を取り込むコマンド (pull.ver)
git pull origin main
```

```
# ローカルの変更を残しつつ、GitHub の更新を取り込むコマンド (rebase.ver)
git pull --rebase origin main
```

VPNサービスを使って、ラズパイのIPアドレスを固定化する方法（Tailscaleを使う方法）

0. そもそも

前述のとおり、WindowsPCやAndroid端末は、mDNSが不安定なので、ラズパイとのSSH接続にはIPアドレスが必要

仮想VPNサービス（ここではTailscale）を使えば

Tailscaleに登録された各デバイスは：

- 固定の仮想IPアドレスを持つ（100.x.y.z 形式）
- デバイスがオンラインの間、そのIPは常に同じ
- 管理画面に表示される
- そのIPで直接SSH接続が可能になる（実際のネットワークは同じでなくてもよい）

```
ssh pi@100.x.y.z
```

1. 構成手順

0. 前提

PC: Windows（Macならそもそもこの問題は起きないので設定不要） スマホ: Android（iPhoneの人も、スマホでターミナル操作をするなら、多分やったほうがいい。）

1. アカウント作成（PCで）

<https://tailscale.com/>

- Google / GitHub / Microsoft などログイン
- これが 仮想LAN になる

2. Windows にインストール

<https://tailscale.com/download>

- Windows版をDL
- インストール
- ログイン
- Tailscale はタスクトレイ常駐アプリとしてふるまう。

3. スマホ にも入れる

- デスクトップで表示されるQRコードか Playストア で検索してインストール
- ログイン
- デスクトップに端末が追加されたか確認
- 案内されるテストコマンドをPCで実行して接続を確認できる

```
ping 100.x.y.z
```

4. Raspberry Pi にもインストール

ラズパイで：

```
curl -fsSL https://tailscale.com/install.sh | sh
```

終わったら：

```
sudo tailscale up
```

すると、URLが出るので、**PCで開いてログイン**。

5. ここまでで何が起きているか

この時点で：

- Windows
- Android
- Raspberry Pi

が **同じ仮想LAN** に入る

6. ラズパイの固定IPを確認する

ラズパイで：

```
tailscale ip -4
```

例：

```
100.64.12.34
```

これがTailscaleで表示される内容と一致するか確認する

7. SSH接続

Windowsから：

```
ssh pi@100.64.12.34
```

さらに便利なところ

ホスト名でSSHできるようになる

Tailscale管理画面に行くと：

`raspberrypi.tailnet-name.ts.net`

みたいな名前が付くので、

これで

```
ssh pi@raspberrypi
```

も可能になる（Windows PowerShellでOK）。

再起動時に自動接続

通常は自動で再接続されるが、念のため

```
sudo tailscale set --auto-update
```

Raspberry Pi Zero 2 W の起動時にIPアドレスを任意のDiscordサーバーに送信させるようにする方法（シェルスクリプトの方法）

0. Discordでウェブフックのリンクを取得

- Discordにログインし、ウェブフックを作成したいサーバーを選択
- チャンネルの"Server Settings"を開き、"Integrations"タブを選択し、"Webhooks"をクリック
- "New Webhook"をクリックし、ウェブフックの名前やアイコンを設定。
- ウェブフックURLをコピー

1. スクリプトファイルを作成

```
nano ~/discord_ip.sh
```

2. 以下のコードを貼り付け

```
#!/bin/bash
```

```
WEBHOOK_URL="ここにコピーしたURLを貼り付ける"

# IPアドレス取得関数 (Google DNSへのルート情報から取得)
get_ip() {
    ip route get 8.8.8.8 | grep -oP 'src \K\S+'
}

# 最大10回リトライ
for i in {1..10}; do
    IP_ADDR=$(get_ip)

    if [ -n "$IP_ADDR" ]; then
        # JSONペイロードの作成
        PAYLOAD="{\"content\": \"🚀 起動成功！\\nIPアドレス: \`$IP_ADDR\`\"}"

        # Discordへ送信 (-s: 静かに, -o: 出力なし, -w: ステータスコード表示)
        STATUS=$(curl -H "Content-Type: application/json" -X POST -d "$PAYLOAD" -s
-o /dev/null -w "%{http_code}" "$WEBHOOK_URL")

        if [ "$STATUS" -eq 204 ]; then
            echo "Successfully sent to Discord"
            exit 0
        else
            echo "Post failed with status: $STATUS"
        fi
    fi

    echo "Retry $i..."
    sleep 30
done
```

3. スクリプトに実行権限を付与

```
chmod +x ~/discord_ip.sh
```

4. 試しに実行してみる

```
./discord_ip.sh
```

5. 自動起動の設定をする

ラズパイが電源ONになったとき、このスクリプトを自動で実行するように設定。ここでは crontab を使う。

1. ラズパイのターミナルで以下を実行

```
crontab -e
```

(初めて使う場合は、1番の nano を選択)

2. 一番下の行に、以下の内容を追記 (起動時と5分おきに実行するように設定)

```
@reboot ~/discord_ip.sh  
*/5 * * * * ~/discord_ip.sh
```

ファイルパスは適応書き換え

3. 保存して終了

4. 再起動

```
sudo reboot
```

12. 参考資料

- Raspberry Pi公式ドキュメント: <https://www.raspberrypi.com/documentation/>
- Tailscale公式サイト: <https://tailscale.com/>
- Discordウェブフックドキュメント: <https://discord.com/developers/docs/resources/webhook>
- 設計メモ_TRC2026基板: https://docs.google.com/document/d/1BoxN7ev75-qyxDMI1QDe3UI_lqAFg0KLx-Su-Op7N-4/edit?tab=t.0
- TRC2026 電子部品表: https://docs.google.com/spreadsheets/d/1rFDZrWUXG1Hqm-SPN9i2vzo1shoo_CkjGAdZZAB9toc/edit?gid=1327550036#gid=1327550036